MATH 189 - Homework 6

due Tuesday, March 4 at 8:00AM

Resources: These problems appear in Ch. 10 of the textbook pgs. 465-467.

Format requirements: Type your homeworks using LaTeXwith embedded plots for the applied exercises.

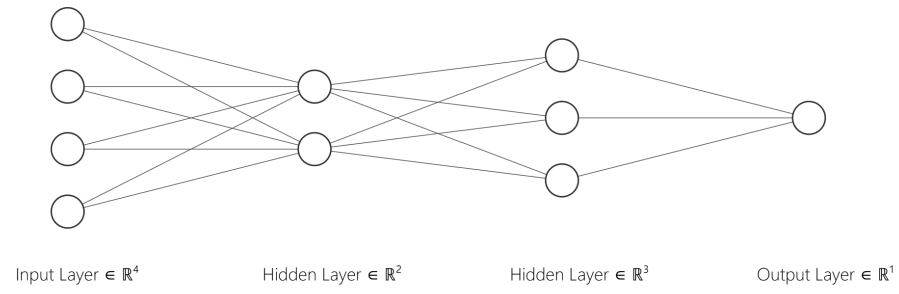
Grading: The grading is based 50% on completion and 50% on correctness.

Problem 1. Ch. 10 conceptual exercise 1

Consider a neural network with two hidden layers: $p = 4$ input units, 2 units in the first hidden layer, 3 units in the second hidden layer, and a single output.

(a) Draw a picture of the network, similar to Figures 10.1 or 10.4 in the textbook.

Solution:



(b) Write out an expression for $f(x)$, assuming ReLU activation functions. Be as explicit as you can!

Solution:

$$f(x) = \max(0, w^T \max(0, W_2 \max(0, W_1 x + b_1) + b_2) + b_3)$$

(c) Now plug in some values for the coefficients and write out the value of $f(x)$.

Solution: Let $x = \begin{bmatrix}1 & 1 & 0 & 0\end{bmatrix}^T$, $W_1 = \begin{bmatrix}1 & 1 & 0 & 0\\ 0 & 0 & 1 & 1\end{bmatrix}$, $b_1 = \begin{bmatrix}0\\0\end{bmatrix}$, $W_2 = \begin{bmatrix}1 & 0\\ 1 & 1\\ 0 & 1\end{bmatrix}$, $b_2 = \begin{bmatrix}-1\\0\\-1\end{bmatrix}$, $w = \begin{bmatrix}1 & 1 & 1\end{bmatrix}^T$, $b_3 = 0$.

$h_1 = \max(0, W_1 x + b_1) = \begin{bmatrix}2 & 0\end{bmatrix}^T$, $h_2 = \max(0, W_2 h_1 + b_2) = \begin{bmatrix}1 & 2 & 0\end{bmatrix}^T$

$f(x) = \max(0, w^T h_2 + b_3) = \boxed{3}$

(d) How many parameters are there?

Solution:

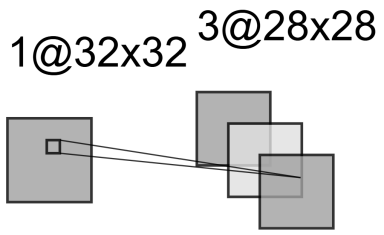
$$(4+1) \times 2 + (2+1) \times 3 + (3+1) \times 1 = 10 + 9 + 4 = 23$$

Problem 2. Ch. 10 conceptual exercise 4

Consider a CNN that takes in 32×32 grayscale images and has a single convolution layer with three 5×5 convolution filters (without boundary padding).

(a) Draw a sketch of the input and first hidden layer similar to Figure 10.8 in the textbook.

Solution:



(b) How many parameters are in this model?

Solution:

$$3 \times (5 \times 5 + 1) = 3 \times 26 = 78$$

(c) Explain how this model can be thought of as an ordinary feedforward neural network with the individual pixels as inputs, and with constraints on the weights in the hidden units. What are the constraints?

Solution: In the feedforward view, inputs are the 1024 pixels. Hidden units are the $3 \times 28 \times 28 = 2352$ outputs (one per filter position). Each hidden unit takes a 5×5 patch of the input (25 pixels).

The constraints are the same 25 weights are used at every spatial position for a given filter. That is, the filter weights are shared across all positions where the filter is applied.

(d) If there were no constraints, then how many weights would there be in the ordinary feed-forward neural network in (c)?

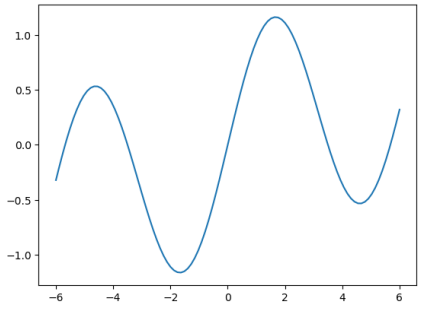
Solution: Without sharing weights, there would be $3 \times 28 \times 28 \times (5 \times 5 + 1) = 61,152$ weights.

Problem 3. Ch. 10 applied exercise 6

Consider the simple function $R(\beta) = \sin(\beta) + \beta/10$.

(a) Draw a graph of this function over the range $\beta \in [-6, 6]$.

Solution:

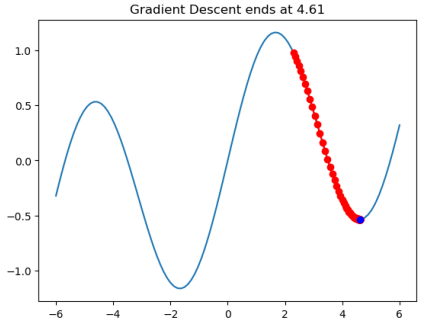


(b) What is the derivative of this function?

Solution: $R'(\beta) = \cos(\beta) + 1/10$

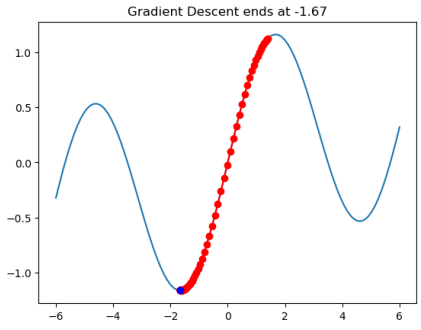
(c) Given $\beta_0 = 2.3$, run gradient descent to find a local minimum of $R(\beta)$ using a learning rate of $\rho = 0.1$. Show each of $\beta_0, \beta_1, \ldots$ in your plot, as well as the final answer.

Solution:



(d) Repeat with $\beta_0 = 1.4$.

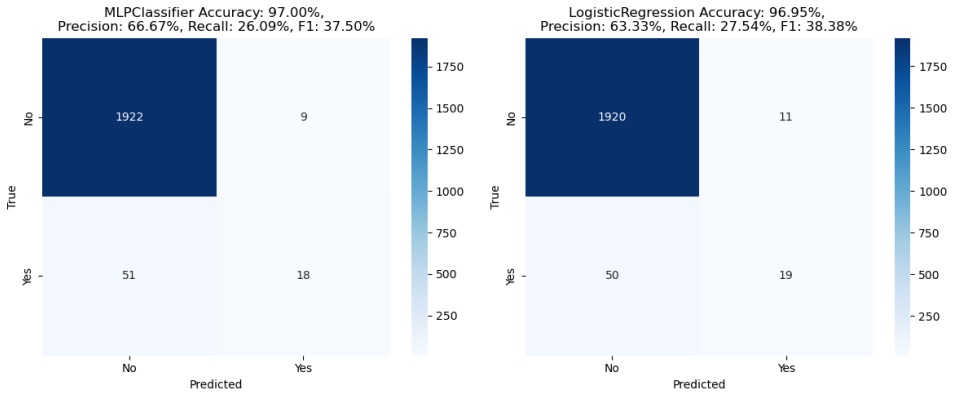
Solution:



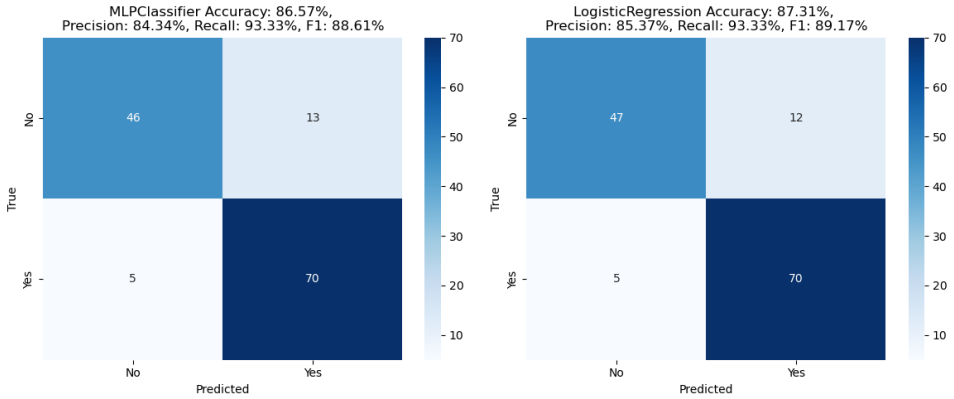
Problem 4. Ch. 10 applied exercise 7 with different regularization

Fit a neural network to the `Default` data. Use a single hidden layer with 10 units and any time of regularization (the book uses dropout regularization, weight decay is more modern). Have a look at Labs 10.9.1-10.9.2 for guidance. Compare the classification performance of your model with that of linear logistic regression.

Solution:



Both models had similarly high accuracy scores and similarly low F1 scores. This is because most of the labels were 0, so the data is imbalanced. With balanced data, the accuracies are slightly lower, but the F1 scores are much higher. Here, they are also almost exactly the same.



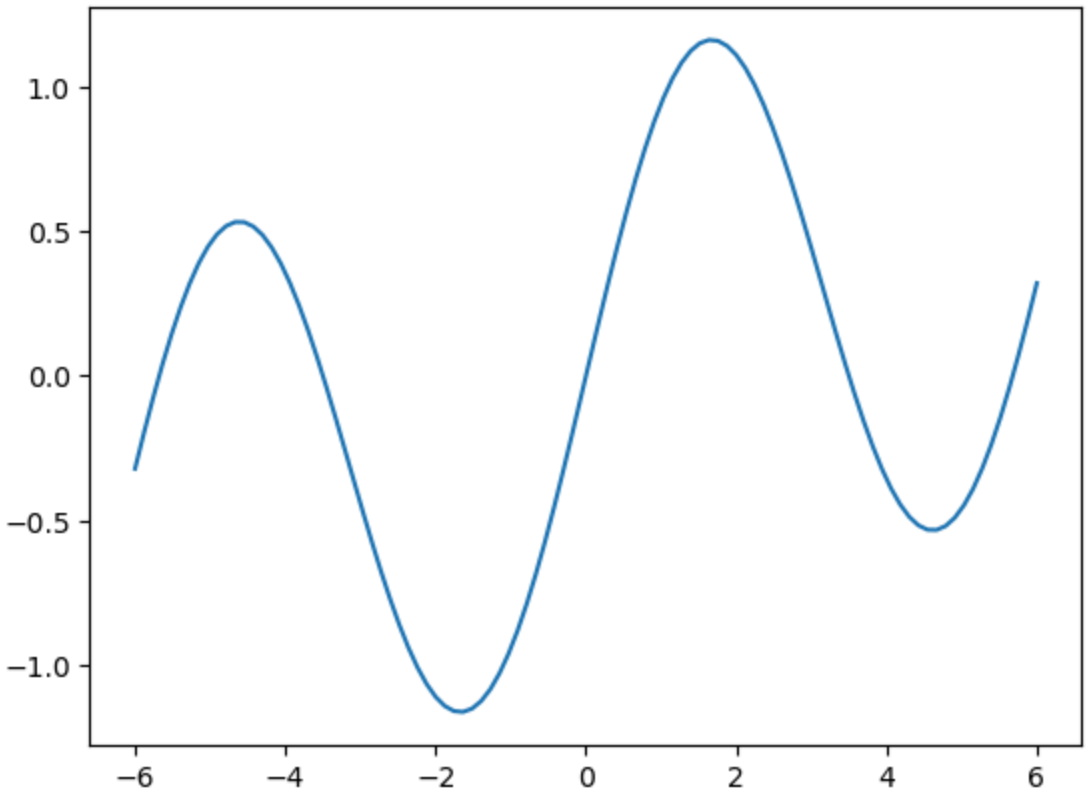
Problem 3

In [52]:
```
import numpy as np
import matplotlib.pyplot as plt
```

In [53]:
```
f = lambda beta: np.sin(beta) + beta / 10
df = lambda beta: np.cos(beta) + 1 / 10
```

In [54]:
```
# 3a
x = np.linspace(-6, 6, 100)
y = f(x)

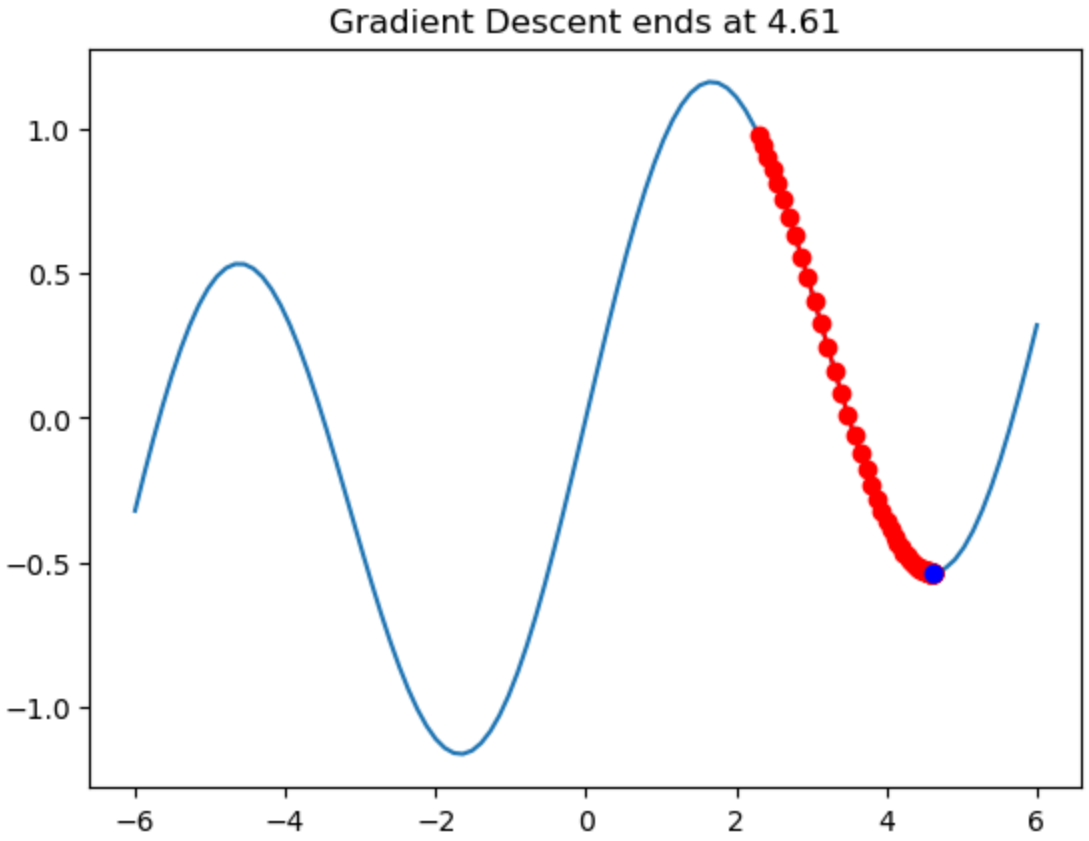
plt.plot(x, y)
plt.show()
```



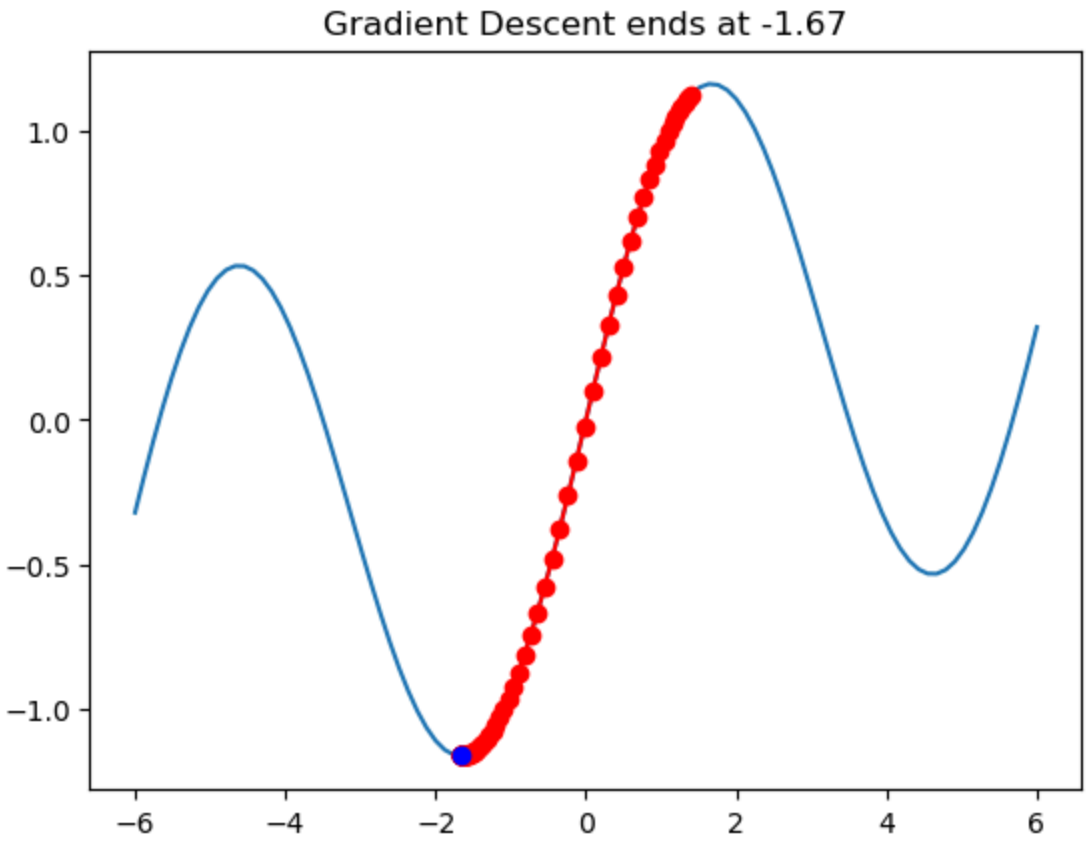
In [55]:
```
def descend(beta, rho = 0.1, num_steps = 100):
    beta_history = [beta]
    for _ in range(num_steps):
        beta = beta - rho * df(beta)
        beta_history.append(beta)

    plt.plot(x, y)
    plt.plot(beta_history, f(np.array(beta_history)), 'ro-')
    plt.plot(beta_history[-1], f(beta_history[-1]), 'bo')
    plt.title(f'Gradient Descent ends at {beta_history[-1]:.2f}')
    plt.show()
```

In [56]:
```
# 3c
descend(2.3)
```



In [57]:
```
# 3d
descend(1.4)
```



Problem 4

In [58]:
```
# Problem 4
from ISLP import load_data
from sklearn.neural_network import MLPClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

data = load_data('Default')
```

In [59]:
```
import seaborn as sns
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

def train_and_plot(X, y):
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

    scaler = StandardScaler()
    X_train_s = scaler.fit_transform(X_train)
    X_test_s = scaler.transform(X_test)

    nn = MLPClassifier(hidden_layer_sizes=(10,), alpha=0.01, max_iter=500, random_s
    lr = LogisticRegression(max_iter=500, random_state=42)

    fig, axes = plt.subplots(1, 2, figsize=(12, 5))
    labels = ['No', 'Yes']
```

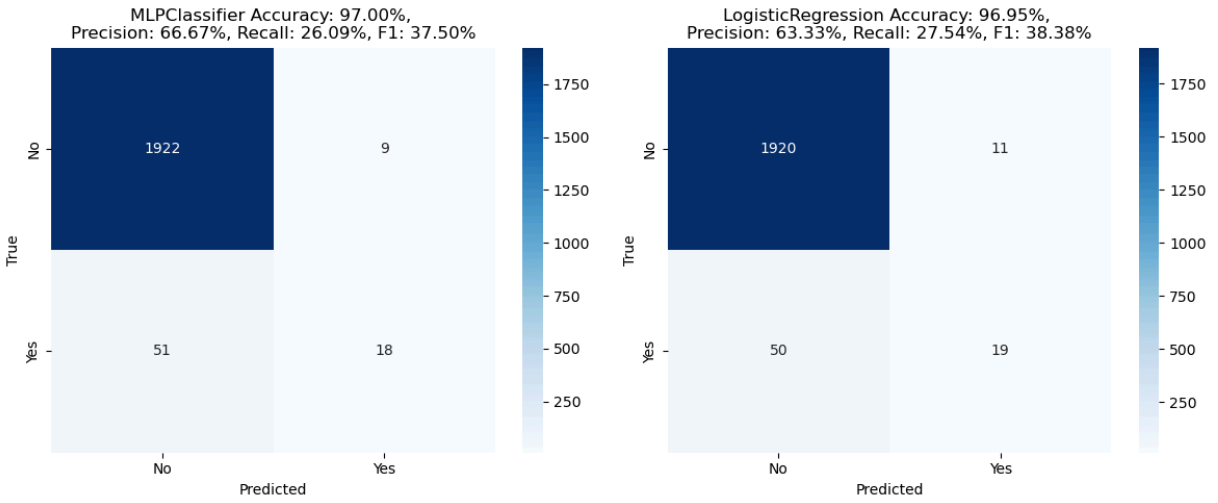
```
    for i, model in enumerate([nn, lr]):
        model.fit(X_train_s, y_train)
        y_pred = model.predict(X_test_s)
        accuracy = accuracy_score(y_test, y_pred)
        precision = precision_score(y_test, y_pred)
        recall = recall_score(y_test, y_pred)
        f1 = f1_score(y_test, y_pred)
        cm = confusion_matrix(y_test, y_pred)

        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[i],
                    xticklabels=labels, yticklabels=labels)
        axes[i].set_title(f'{model.__class__.__name__} Accuracy: {accuracy * 100:.2
        axes[i].set_xlabel('Predicted')
        axes[i].set_ylabel('True')

    fig.tight_layout()
    plt.show()
```

In [60]:
```
y = (data['default'] == 'Yes').astype(int)
X = data[['student', 'balance', 'income']].copy()
X['student'] = (X['student'] == 'Yes').astype(int)

train_and_plot(X, y)
```

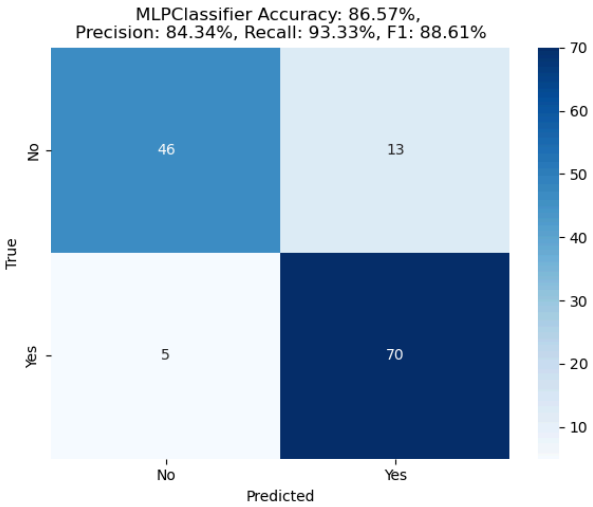


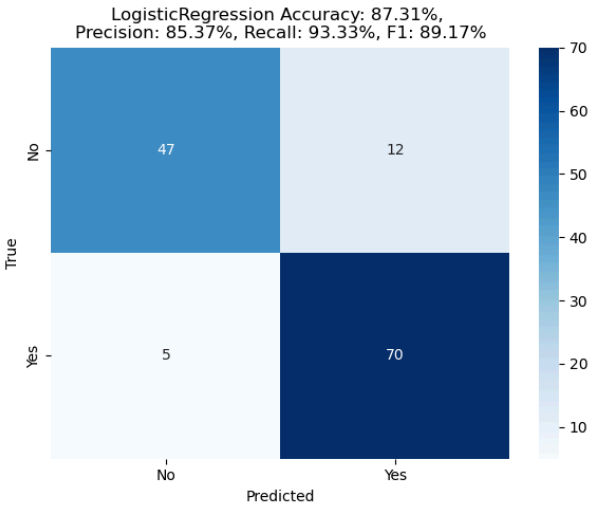
In [61]:
```
import pandas as pd

# balance classes
defaults = data[data['default'] == 'Yes']
non_defaults = data[data['default'] == 'No']
non_defaults = non_defaults.sample(len(defaults), random_state=42)
data_balanced = pd.concat([defaults, non_defaults])

yb = (data_balanced['default'] == 'Yes').astype(int)
Xb = data_balanced[['student', 'balance', 'income']].copy()
Xb['student'] = (Xb['student'] == 'Yes').astype(int)

train_and_plot(Xb, yb)
```


MLPClassifier Accuracy: 86.57%,
Precision: 84.34%, Recall: 93.33%, F1: 88.61%
46
13
5
70
No
Yes
True
No
Yes
Predicted
70
60
50
40
30
20
10


LogisticRegression Accuracy: 87.31%,
Precision: 85.37%, Recall: 93.33%, F1: 89.17%
47
12
5
70
No
Yes
True
No
Yes
Predicted
70
60
50
40
30
20
10